

Lean 4における単純型付きラムダ計算のcall-by-value評価に対する型安全性証明の形式化に関する研究

齋藤 湊馬

Abstract

型付きラムダ計算の型安全性は、closed well-typed term が実行中に stuck しないことを保証する基本的なメタ理論であり、通常 preservation と progress によって示される。本研究では、Lean 4 Metatheory リポジトリに含まれる `STLCextBool` 形式化を対象とし、既存の call-by-value 一段階簡約関係 `CBVStep` に対する型安全性メタ理論を補完する。既存実装には、unrestricted な一段階簡約 `Step` に対する `subject_reduction` と progress、および `CBVStep` の決定性・合流性が含まれていた。本研究では、まず `CBVStep.to_step` として `CBVStep` の任意の一段階簡約が unrestricted `Step` の一段階簡約でもあることを示し、既存の `subject_reduction` を再利用して `CBVStep.preservation` を導く。一方で、unrestricted progress は CBV の評価順序を保証しないため、`CBVStep.progress` は評価順序に沿って別途証明する。さらに、`Rewriting.Star CBVStep` に基づいて `CBVStep.preservation_star` と `CBVStep.type_safety` を形式化する。これにより、既存の `STLCextBool` における call-by-value 操作的意味論に対して、preservation, progress, および no-stuckness に相当する type safety 定理を Lean 4 上で補完した。

1 Introduction

型付きラムダ計算のメタ理論は、プログラミング言語理論における基本的な研究対象である [2]。特に、型安全性は、well-typed な項が何らかの計算を実行中に stuck しないことを保証する性質として重要である。標準的には、型安全性は preservation と progress という二つの定理によって示される。preservation は、ある型を持つ項が一段階簡約された時、簡約後の項も同じ型を持つことを主張する。一方、progress は、closed な well-typed term が値であるか、あるいは次の計算ステップに進めることを主張する。

このようなメタ理論を定理証明支援系によって機械的に検証することは、型システムや操作的意味論に関する証明を厳密に扱うための有効な方法である [3, 6]。紙での数学的な証明は省略されることの多い場合分けや補題の依存関係も、形式化では明示的に扱う必要がある。そのため、形式化を通じて、定理そのものだけでなく、証明がどの既存結果に依存しているのかも明確になる。

本研究では、`Arthur742Ramos/Metatheory` リポジトリに含まれる `STLCextBool` 形式化を対象とする [4, 5]。 `STLCextBool` は、単純型付きラムダ計算に真偽値と条件分岐を加えた体系であり、型、項、型付け関係、簡約関係、およびそれらに関するメタ理論が Lean 4 で形式化されている。この形式化には、unrestricted な一段階簡約関係 `Step` と、call-by-value の一段階簡約関係 `CBVStep` が既に定義されている。

既存の `STLCextBool` 形式化には、unrestricted `Step` に対する `subject_reduction` と progress が含まれている。また、`CBVStep` については、値が一段階簡約できないこと、決定性、および合流性に関する結果が既に与えられている。一方で、既存の `CBVStep` に対する preservation, progress, および multi-step に基づく type safety は、明示的な形では補完の余地があった。特に、unrestricted `Step` に対する progress は、項が何らかの unrestricted な簡約に進めることを示すだけであり、call-by-value の評価順序に従って進めることまでは保証しない。

本研究の目的は、この既存の **CBVStep** に対して call-by-value 型安全性メタ理論を補完することである。まず、**CBVStep** の任意の一段階簡約が **unrestricted Step** の一段階簡約でもあることを示す。これにより、既存の **unrestricted Step** に対する **subject_reduction** を再利用して、**CBVStep** に対する preservation を導く。一方で、progress については、call-by-value の評価順序に沿って別途証明する。さらに、汎用の reflexive-transitive closure である **Rewriting.Star** を用いて、**CBVStep** の multi-step preservation と type safety を形式化する。これにより、closed な well-typed term から **CBVStep** によって任意のステップ数だけ簡約された項は、値であるか、さらに **CBVStep** によって一段階簡約できることを示す。

本稿の構成は次の通りである。第2節では、型付きラムダ計算、操作的意味論、preservation, progress, type safety の基本事項を確認する。第3節では、対象とする **STLCextBool** 形式化と既存結果を整理する。第4節では、本研究で追加した **CBVStep** に関する主要定理を述べる。第5節では、証明構造、既存結果の再利用、本研究の範囲と限界について議論する。第6節で結論と今後の課題を述べる。

2 Background

本節では、本論文で用いる標準的な背景事項を整理する。まず、単純型付きラムダ計算とその拡張について述べる。次に、一段階簡約および多段階簡約による small-step operational semantics を確認する。そのうえで、値と評価順序、特に call-by-value 評価における制約を説明する。最後に、型安全性を構成する preservation と progress, および Lean 4 による mechanized metatheory の位置づけを述べる。

2.1 Simply Typed Lambda Calculus

単純型付きラムダ計算 (Simply Typed Lambda Calculus, STLC) は、型付き計算体系の基本的な例である。STLC では、型、項、文脈、および型付け判断が定義される。型付け判断は通常

$$\Gamma \vdash M : A$$

の形で表され、文脈 Γ の下で項 M が型 A を持つことを意味する。文脈は、変数とその型の対応を保持するものであり、例えば $x : A, y : B$ という文脈の下で項 M が型 C を持つことを、

$$x : A, y : B \vdash M : C$$

とかく。

STLC の中心的な構成は、関数型とラムダ抽象である。関数型 $A \rightarrow B$ は、型 A の項を受け取り、型 B の項を返す関数の型を表す。これに対応する項の構成として、ラムダ抽象と関数適用がある。ラムダ抽象 $\lambda x.M$ は、変数 x を受け取り M を返す関数を表す。一方、関数適用 $M N$ は、関数を表す項 M を引数 N に適用する項である。

ラムダ抽象と関数適用の型付けは、関数型に対応している。例えば、文脈 Γ に型 A の変数 x を追加した文脈の下で、項 M が型 B を持つならば、ラムダ抽象 $\lambda x.M$ は型 $A \rightarrow B$ を持つ。これは次のように表される。

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x.M : A \rightarrow B}$$

また、項 M が型 $A \rightarrow B$ を持ち、項 N が型 A を持つならば、関数適用 $M N$ は型 B を持つ。

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B}$$

本論文では、この STLC に真偽値と条件分岐を加えた体系である **STLCextBool** を対象とする。**STLCextBool** には、真偽値型 **Bool**、真を表す項 **true**、偽を表す項 **false**、および条件分岐項

$$\text{if } M \text{ then } N_1 \text{ else } N_2$$

が含まれる。条件分岐項は、条件部 M が型 `Bool` を持ち、二つの分岐 N_1 と N_2 が同じ型を持つときに型付けされる。すなわち、条件分岐の型付けは次の形で表される。

$$\frac{\Gamma \vdash M : \text{Bool} \quad \Gamma \vdash N_1 : A \quad \Gamma \vdash N_2 : A}{\Gamma \vdash \text{if } M \text{ then } N_1 \text{ else } N_2 : A}$$

本研究では、**STLCextBool** に既存で定義されている call-by-value 一段階簡約関係 **CBVStep** を対象として、preservation, progress, および type safety を補完する。

2.2 Small-step Operational Semantics

操作的意味論 (operational semantics) は、項がどのように計算されるかを記述するための意味論である。特に small-step operational semantics では、計算を一度に最終結果まで進めるのではなく、一回のステップごとに記述する。一段階簡約関係を、

$$M \longrightarrow N$$

と書くとき、これは項 M が一回の計算ステップによって項 N に簡約されることを意味する。

一段階簡約に対して、0 回以上の簡約を表す多段階簡約も重要である。多段階簡約は、一段階簡約を反射的かつ推移的に閉じた関係として定義される。すなわち、任意の項 M は 0 回の簡約で自分自身に到達可能である。また、 M が N に多段階で到達し、さらに N が P に一段階簡約が可能であるならば、 M は P に多段階簡約できる。本論文で対象とする Lean 4 形式化では、このような反射推移閉包を **Rewriting.Star** で表す。

本論文で扱う **STLCextBool** 形式化には、unrestricted な一段階簡約関係 **Step** と、call-by-value の一段階簡約関係 **CBVStep** が含まれている。unrestricted **Step** は、評価順序を call-by-value に固定しない一段階簡約の関係である。一方、**CBVStep** は call-by-value 評価順序に従う一段階簡約関係であり、関数適用や条件分岐において、どの部分項を先に評価するかが制限されている。

例えば、関数適用 $M N$ の call-by-value 評価では、まず関数部分 M を評価する。関数部分 M が値になった後で、引数部分 N を評価する。そして、関数部分がラムダ抽象であり、引数部分も値である場合にのみ、 β 簡約を行う。このように、call-by-value 簡約では、単に簡約可能な部分項を任意に選ぶのではなく、評価順序に従って一段階ずつ簡約を行う。

したがって、unrestricted **Step** に対する性質がそのまま **CBVStep** に対して成り立つとは限らない。特に、unrestricted **Step** に対する progress は、ある項が何らかの unrestricted な一段階簡約に進めることを示しているのみであり、その簡約が call-by-value の評価順序に従っているかどうかは、保証されていない。この違いは、本研究で **CBVStep** に対する progress を別途証明する理由の一つである。

2.3 Values and Evaluation Order

値 (value) は、それ以上計算を進める必要のない項を表す。call-by-value 評価では、関数適用において、引数を値まで評価してから関数本体への代入を行う。そのためどの項を値とみなすか、またどの順序で部分項を評価するかが重要になる。

本論文で対象とする **STLCextBool** では、ラムダ抽象、true, false が値として扱われる。すなわち、ラムダ抽象 $\lambda x.M$ は関数を表す値であり、真偽値定数である true と false もそれ以上簡約されない値である。Lean 4 形式化では、これらの値は **IsValue** によって表される。

call-by-value における関数適用 $M N$ の評価は、次の順序に従う。まず、関数部分 M を評価する。関数部分 M が値になった後で、引数部分 N を評価する。そして、 M がラムダ抽象であり、かつ N が値である場合にのみ、 β 簡約を行う。この制約により、call-by-value 簡約では、関数本体への代入が行われる前に、引数が必ず値になっている。

条件分岐

$$\text{if } M \text{ then } N_1 \text{ else } N_2$$

の評価では、まず条件部 M を評価する。条件部が `true` であれば `then` 分岐 N_1 に簡約され、条件部が `false` であれば `else` 分岐 N_2 に簡約される。条件部がまだ値でない場合には、`then` 分岐や `else` 分岐を先に評価するのではなく、条件部の評価を進める。

このように、`call-by-value` 評価では、値と評価順序が密接に関係している。unrestricted な簡約関係では、簡約可能な部分項をより自由に選ぶことができるが、`call-by-value` 簡約では、評価できる位置や順序が制限される。したがって、closed な well-typed term が何らかの unrestricted な簡約に進めることを示すだけでは、`call-by-value` の progress を示したことにはならない。本研究で証明する **CBVStep.progress** は、この評価順序に従って、closed な well-typed term が値であるか、あるいは **CBVStep** によって一歩進めることを主張する。

2.4 Type Safety: Preservation and Progress

型安全性 (type safety) は、closed well-typed な項が計算の途中で stuck しないことを保証する性質である [7, 2]。ここで stuck とは、値ではない状態のまま、次の計算ステップに進めない状態を指す。型安全性は、通常の場合、preservation と progress という二つの定理を組み合わせて示される。

preservation は、項の型が簡約によって保存されることを主張する。すなわち、文脈 Γ の下で項 M が型 A を持ち、かつ M が一段階簡約で N に進むならば、 N も文脈 Γ の下で型 A を持つ。これは次のように表される。

$$\text{if } \Gamma \vdash M : A \text{ and } M \longrightarrow N, \text{ then } \Gamma \vdash N : A.$$

この性質は、計算を一段階進めた場合に型付けが壊れないことを表している。

progress は、closed な well-typed term が、すでに値であるか、あるいは一段階計算を進められることを主張する。closed であるとは、項が自由変数を持たないことを意味する。したがって progress は、通常、空文脈の下で型付けされた項に対して述べられる。その標準的な形は次のようになる。

$$\text{if } \emptyset \vdash M : A, \text{ then } M \text{ is a value or there exists } N \text{ s.t. } M \longrightarrow N$$

これは、closed な well-typed term が、値でないならば計算を進めることができ、次のステップ N が存在することを意味する。

preservation と progress を組み合わせることで、closed な well-typed な項から計算を始めたとき、計算の各段階で型が保存され、さらに値でない限り次のステップに進めることができる。したがって well-typed な項は、値でない場合には計算を進められない stuck state には到達しない。この意味で、preservation と progress は type safety を構成する標準的な二つの要素である。

本論文では、この標準的な型安全性の形を、**STLCextBool** に既存で定義されている `call-by-value` 一段階簡約関係 **CBVStep** に対して補完する。具体的には、**CBVStep** に対する一段階の preservation と progress を示し、さらに **Rewriting.Star** によって表される、多段階の **CBVStep** に対して preservation を示す。その上で、closed な well-typed term から **CBVStep** で到達可能な任意の項が、値であるか、あるいはさらに一段階 **CBVStep** で進めることを type safety として形式化する。

2.5 Mechanized Metatheory in Lean 4

Mechanized metatheory とはプログラミング言語の構文、型付け規則、操作的意味論、およびそれらに関するメタ理論を、定理証明支援系を用いて機械的に検証する研究である。代表的な教材・形式化として、Coq/Rocq を用いる Software Foundations や Agda を用いる PLFA がある [3, 6]。紙上の証明では、場合分けや補題の適用が省略されることが多い。一方、Lean 4 のような定理証明支援系では、定義、補題、定理、およびそれらの依存関係を明示的

に与える必要がある [1]. そのため, 形式化は, 証明の正当性を機械的に検証するだけでなく, 証明がどの定義や補題に依存しているのかを明確にする役割も持つ.

Lean 4 では, 型や項の構文, 型付け関係, 簡約関係を帰納的定義として表現できる. また, `preservation` や `progress` のようなメタ理論も, Lean 4 の定理として記述し, 証明することができる. 証明が `sorry` なしで完了し, 対象モジュールがビルドできることにより, 定理が Lean 4 のカーネルによって検証されたことを確認できる.

本論文では, Lean 4 上の定義名や定理名を, `CBVStep.progress` や `Rewriting.Star` のようにコード表記で引用する. これにより, 本文中の数学的主張と, 実際に形式化された Lean 4 の定義, 定理との対応を明確にする. 本研究の主な追加定理は次の通りである.

```
CBVStep.to_step,      CBVStep.preservation,
CBVStep.progress,    CBVStep.preservation_star,
CBVStep.type_safety.
```

3 Existing Formalization

本節では, 本研究が対象とする既存の `STLCextBool` 形式化を整理する. この形式化は, Lean 4 の Metatheory リポジトリに含まれている [4, 5]. 特に, 型, 項, 型付け関係, 簡約関係, 値, および既存の `CBVStep` に関する結果を確認する. 本節の目的は, 既存の形式化に含まれていた要素と, 本研究で補完する要素を明確に区別することである.

本論文では, 数学的な説明では標準的な名前付き記法を用い, Lean 4 上の定義や定理を述べる際には実装上の表現に対応する定理名を用いる. したがって, 背景説明では $\Gamma \vdash M : A$ のような標準的な型付け判断を用いるが, Lean 4 上の対象を指すときには `Term`, `Ty`, `HasType` などの定義名を用いる.

3.1 Terms, Types, and Typing in STLCextBool

本研究で対象とする `STLCextBool` 形式化では, 型付きラムダ計算の構文と型付け関係が Lean 4 で定義されている. 主な対象は, 型を表す `Ty`, 項を表す `Term`, 文脈を表す `Context`, および型付け関係を表す `HasType` である. これらの定義は `Metatheory/STLCextBool/` 以下に分かれており, `Ty` は `Types.lean`, `Term` は `Terms.lean`, `Context` と `HasType` は `Typing.lean` で定義されている.

数学的には, 型付け関係は,

$$\Gamma \vdash M : A$$

という形で表される. これは, 文脈 Γ の下で項 M が型 A を持つことを意味する. Lean 4 上では, この判断に対応する関係が, `HasType` として形式化されている. 本論文では, 数学的な説明では上記のような標準的な記法を用い, Lean 4 上の定義や定理を述べる際には `HasType` に基づく定理名を参照する.

`STLCextBool` は, 単純型付きラムダ計算に真偽値と条件分岐を加えた体系である. そのため関数型とラムダ抽象, 関数適用に加えて, 真偽値型, 真を表す項, 偽を表す項, および条件分岐項が含まれる. 数学的には, 真偽値型を `Bool`, 真偽値定数を `true` および `false` と書く. 条件分岐項は

$$\text{if } M \text{ then } N_1 \text{ else } N_2$$

の形で表され, 条件部 M が型 `Bool` を持ち, 二つの分岐 N_1 と N_2 が同じ型を持つときに型付けされる.

この型付け規則は, 標準的には次の形で表される.

$$\frac{\Gamma \vdash M : \text{Bool} \quad \Gamma \vdash N_1 : A \quad \Gamma \vdash N_2 : A}{\Gamma \vdash \text{if } M \text{ then } N_1 \text{ else } N_2 : A}$$

この規則は、条件部が真偽値として評価され、どちらの分岐に進んでも同じ型 A の項が得られることを保証している。

以上のように、**STLCextBool** 形式化は、項、型、文脈、型付け関係を Lean 4 上で明示的に定義している。本研究では、この既存の型付け関係を前提として、後述する call-by-value 一段階簡約関係 **CBVStep** に対する preservation, progress, および type safety を形式化する。

3.2 Unrestricted Reduction

STLCextBool 形式化には、unrestricted な一段階簡約関係 **Step** が既存で定義されている。これは、項が一回の計算ステップによって別の項へ簡約されることを表す関係である。数学的には、一段階の簡約を

$$M \longrightarrow N$$

のように表す。本論文では、この unrestricted な一段階簡約に対応する Lean 4 上の関係を **Step** と呼ぶ。

Step には、関数適用に対する β 簡約や、条件分岐に対する簡約規則が含まれている。例えば、ラムダ抽象を引数に適用する項は、関数本体への代入によって簡約される。また、条件分岐では、条件部が true であれば then 分岐へ、false であれば else 分岐へ簡約される。さらに、関数適用や条件分岐の部分項に対しても簡約を進めるための規則が含まれている。

既存定理 **subject_reduction** は、この unrestricted **Step** に対する preservation を与える。これは、項 M が型 A を持ち、かつ M が **Step** によって N に一段階簡約されるならば、簡約後の項 N も同じ型 A を持つことを主張する。数学的には、以下の形で表される。

$$\text{if } \Gamma \vdash M : A \text{ and } M \longrightarrow N, \text{ then } \Gamma \vdash N : A.$$

また、多段階の unrestricted 簡約に対する preservation も **subject_reduction_multi** として既存で与えられている。さらに、既存の形式化には、unrestricted **Step** に対する progress も含まれている。これは、closed な well-typed term が、値であるか、あるいは unrestricted **Step** によって一段階簡約できることを主張する。すなわち、空文脈の下で $\vdash M : A$ が成り立つならば、 M は値であるか、ある項 N が存在して $M \longrightarrow N$ となる。

ただし、unrestricted **Step** は、call-by-value の評価順序そのものを表す関係ではない。例えば、**Step** の β 簡約は、引数が値であることを要求しない。また、関数適用の右辺や条件分岐の各部分項も、call-by-value の評価順序とは独立に簡約できる。そのため、unrestricted **Step** に対する progress は、項が何らかの一段階簡約に進めることを示すが、その簡約が call-by-value 評価順序に従っていることは保証しない。したがって、既存の unrestricted progress から、**CBVStep** に対する progress を直接得ることはできない。この点が、本研究で **CBVStep.progress** を別途証明する必要がある理由である。

3.3 Values and Canonical Forms

STLCextBool 形式化には、値を表す述語 **IsValue** が既に定義されている。値とは、call-by-value 評価ではそれ以上評価しない停止点として扱われる項である。本体系では、ラムダ抽象、真、偽が値である。なお、unrestricted **Step** ではラムダ本体の簡約を許すため、ここでの停止性は **CBVStep** に関するものである。

値に関する重要な補題として、canonical forms がある。canonical forms は、closed な well-typed term が、その型に応じてどのような形をしているかを述べる補題である。**STLCextBool** 形式化には、関数型と真偽値型に対する canonical forms が既存補題として与えられている。まず、closed な項 V が関数型 $A \rightarrow B$ をもち、かつ V が値であるならば、 V はラムダ抽象の形をしている。この事実は、Lean 4 形式化では **canonical_forms_arr** に対応する。

Lemma 3.1 (Canonical forms for functions, existing).

$\emptyset \vdash V : A \rightarrow B$ かつ V が **IsValue** を満たすならば, V はラムダ抽象である.

同様に, closed な項 V が型 **Bool** を持ち, かつ V が値であるならば, V は **true** または **false** のいずれかである. この事実は, Lean 4 形式化では **canonical_forms_bool** に対応する.

Lemma 3.2 (Canonical forms for booleans, existing).

$\emptyset \vdash V : \text{Bool}$ かつ V が **IsValue** を満たすならば, $V = \text{true}$ または $V = \text{false}$ である.

これらの canonical forms は, 型付けされた値の形を型から復元するために用いられる. 本研究で補完する **CBVStep.progress** の証明では, これらの canonical forms が重要になる. 例えば, 関数適用 $M N$ の progress を示す場合, まず関数部分 M が値であるか, あるいは **CBVStep** によって一歩進めるかを調べる. M が値であり, かつ関数型を持つ場合には, 補題 3.1 によって M がラムダ抽象であることが分かる. そのうえで, 引数 N が値であれば call-by-value の β 簡約を行うことができる.

また, 条件分岐

if M then N_1 else N_2

の progress を示す場合, 条件部 M が値であり, かつ型 **Bool** を持つならば, 補題 3.2 によって M は **true** または **false** のいずれかであることが分かる. したがって, 条件部が **true** の場合には then 分岐へ, **false** の場合には else 分岐へ **CBVStep** によって進めることができる.

このように, 既存の **IsValue**, **canonical_forms_arr**, および **canonical_forms_bool** は, 本研究で追加する **CBVStep.progress** の証明において, 型付けされた値の形を解析するための基礎として用いられる.

3.4 Call-by-Value Reduction

STLCextBool 形式化には, call-by-value の一段階簡約関係 **CBVStep** が既存で定義されている. **CBVStep** は, unrestricted な一段階簡約関係 **Step** とは異なり, call-by-value の評価順序に沿って項を一段階だけ簡約する関係である. 数学的には, 項 M が call-by-value によって項 N へ一段階簡約されることを

$$M \longrightarrow_{\text{cbv}} N$$

のように表す. 本論文では, この関係に対応する Lean 4 上の既存定義を **CBVStep** と呼ぶ.

CBVStep には, 関数適用に関する規則と条件分岐に関する規則が含まれている. 関数適用については, 関数部分を先に評価し, 関数部分が値になった後で引数部分を評価する. そして, 関数部分がラムダ抽象であり, 引数部分が値である場合にのみ, call-by-value の β 簡約を行う. この点で, **CBVStep** の β 規則は引数が値であることを要求する.

既存の **CBVStep** における関数適用の規則は, 概念的には次のように表される.

Definition 3.3 (Call-by-value rules for application, existing).

関数適用に関する **CBVStep** の規則は, 次の三つである.

$$\frac{M \longrightarrow_{\text{cbv}} M'}{M N \longrightarrow_{\text{cbv}} M' N}$$

$$\frac{\text{IsValue}(M) \quad N \longrightarrow_{\text{cbv}} N'}{M N \longrightarrow_{\text{cbv}} M N'}$$

$$\frac{\text{IsValue}(V)}{(\lambda x. M) V \longrightarrow_{\text{cbv}} M[x := V]}$$

これらは Lean 4 形式化では, それぞれ **CBVStep.appL**, **CBVStep.appR**, および **CBVStep.beta** に対応する.

第一の規則 **CBVStep.appL** は、関数適用 $M N$ の関数部分 M が call-by-value で一步進めるならば、適用全体も関数部分を一步進められることを表す。第二の規則 **CBVStep.appR** は、関数部分 M がすでに値であり、引数部分 N が一步進めるならば、引数部分を一步進めることを表す。第三の規則 **CBVStep.beta** は、関数部分がラムダ抽象であり、引数が値であるときに、関数本体へ引数を代入する call-by-value の β 簡約を表す。ここで重要なのは、**CBVStep.beta** が unrestricted な β 簡約とは異なり、引数が値であることを要求する点である。

条件分岐については、まず条件部を評価する。条件部が true であれば then 分岐へ簡約され、条件部が false であれば else 分岐へ簡約される。条件部がまだ値でない場合には、then 分岐や else 分岐を先に評価するのではなく、条件部の評価を進める。

既存の **CBVStep** における条件分岐の規則は、概念的には次のように表される。

Definition 3.4 (Call-by-value rules for conditionals, existing).

条件分岐に関する **CBVStep** の規則は、次の三つである。

$$\begin{aligned} & \text{if true then } N_1 \text{ else } N_2 \longrightarrow_{\text{cbv}} N_1 \\ & \text{if false then } N_1 \text{ else } N_2 \longrightarrow_{\text{cbv}} N_2 \\ & \frac{M \longrightarrow_{\text{cbv}} M'}{\text{if } M \text{ then } N_1 \text{ else } N_2 \longrightarrow_{\text{cbv}} \text{if } M' \text{ then } N_1 \text{ else } N_2} \end{aligned}$$

これらは Lean 4 形式化では、それぞれ **CBVStep.iteTrue**, **CBVStep.iteFalse**, および **CBVStep.iteC** に対応する。

このように、**CBVStep** は、関数適用と条件分岐に対して call-by-value の評価順序を明示的に反映した一步簡約関係である。特に、関数適用では関数部分を先に評価し、関数部分が値になってから引数部分を評価する。また、 β 簡約は引数が値である場合にのみ行われる。条件分岐では、分岐先を先に評価するのではなく、条件部を先に評価する。

本研究では、この既存の **CBVStep** を新しく定義するのではない。本研究の目的は、既存の **CBVStep** に対して、preservation, progress, multi-step preservation, および type safety を補完することである。そのため、後の主要結果では、ここで整理した既存の **CBVStep** の各規則を用いて、**CBVStep** が unrestricted **Step** に埋め込めること、および closed な well-typed term が **CBVStep** に関して progress を満たすことを示す。

3.5 Existing Results

STLCextBool 形式化には、前節までで述べた構文、型付け関係、値、unrestricted **Step**, および call-by-value 一步簡約関係 **CBVStep** に加えて、いくつかのメタ理論的結果が既存で含まれている。本節では、これらの既存結果を整理し、本研究で補完する結果との境界を明確にする。

まず、**CBVStep** に関する基本的な性質として、値は call-by-value によって一步簡約できないことが既存で示されている。この結果は Lean 4 形式化では **CBVStep.value_no_step** として与えられている。概念的には、項 V が値であるならば、 V から **CBVStep** によって進む項は存在しない、という性質である。

Lemma 3.5 (Values do not step, existing).

V が **IsValue** を満たすならば、 $V \longrightarrow_{\text{cbv}} N$ となる項 N は存在しない。

この補題は、call-by-value 評価において値が計算結果であることを表している。すなわち、値はそれ以上評価される項ではなく、評価の停止点として扱われる。

次に、**CBVStep** の決定性が既存で示されている。決定性とは、同じ項から call-by-value によって一步簡約できる先が高々一つであることを意味する。Lean 4 形式化では、この性質は **CBVStep.deterministic** および **cbv_deterministic** として与えられている。概念的には、次のように表される。

Lemma 3.6 (Determinism of **CBVStep**, existing).

$M \rightarrow_{\text{cbv}} N_1$ かつ $M \rightarrow_{\text{cbv}} N_2$ ならば, $N_1 = N_2$ である.

この性質は, **CBVStep** が評価順序を一意に定める簡約関係であることを表している. **unrestricted** な簡約関係では, 複数の部分項が同時に簡約可能である場合, どの部分項を先に簡約するかによって異なる一歩簡約先が現れることがある. 一方, **call-by-value** 評価では, 評価すべき位置が規則によって制限されているため, 一歩簡約先が一意に定まる.

さらに, 既存の形式化には **CBVStep** に関する合流性も含まれている. この結果は Lean 4 形式化では **cbv_confluent** として与えられている. 合流性は, ある項から複数の簡約列で異なる項に到達したとしても, さらに簡約を進めることで共通の項に到達できることを表す性質である.

Lemma 3.7 (Confluence of call-by-value reduction, existing).

M から **CBVStep** の多段階簡約によって N_1 と N_2 に到達できるならば, ある項 P が存在して, N_1 と N_2 はそれぞれ **CBVStep** の多段階簡約によって P に到達できる.

このように, 既存の **STLCextBool** 形式化には, **CBVStep** そのものに加えて, 値が一歩簡約しないこと, 決定性, および合流性に関する結果が含まれていた. 一方で, 既存の **CBVStep** に対する **preservation**, **progress**, **Rewriting.Star** に基づく **multi-step preservation**, および **type safety** は, 本研究で補完する対象である.

したがって, 本研究の貢献は, **CBVStep** を新たに定義することではない. 本研究の貢献は, 既存の **call-by-value** 一歩簡約関係 **CBVStep** とその既存結果を前提として, 型安全性に関するメタ理論を補完する点にある. 具体的には, 次節で述べるように, **CBVStep** が **unrestricted Step** に埋め込めることを示し, 既存の **subject_reduction** を再利用して **CBVStep** に対する **preservation** を導く. さらに, **call-by-value** 評価順序に沿った **progress** を別途証明し, **Rewriting.Star** に基づく **multi-step preservation** と **type safety** を形式化する.

4 Main Results

本節では, 本研究で追加した **CBVStep** に関する主要定理を述べる. 対象とする **CBVStep** は, **STLCextBool** 形式化に既存で定義されていた **call-by-value** 一段階簡約関係である. 本研究では, この既存の **CBVStep** に対して, **unrestricted Step** への埋め込み, 一段階の **preservation**, **progress**, 多段階の **preservation**, および **type safety** を形式化した.

以下では, 数学的な説明において, **CBVStep** による一段階簡約を,

$$M \rightarrow_{\text{cbv}} N$$

と書く. また, **Rewriting.Star CBVStep M N** に対応する多段階簡約を

$$M \rightarrow_{\text{cbv}}^* N$$

と書く.

Lean 4 上の定理 statement は, 必要に応じて **lstlisting** 環境で示す.

4.1 Embedding CBVStep into Step

最初の結果は, **CBVStep** による任意の一段階簡約が, **unrestricted Step** による一段階簡約でもあることを示すものである. この結果は, 既存の **call-by-value** 簡約関係 **CBVStep** と, 既存の **unrestricted** 簡約関係 **Step** を接続するための基本的な補題である. Lean 4 形式化では, この結果は **CBVStep.to_step** として与えられる.

```
theorem to_step {M N : Term} :
  M →cbv N → Step M N
```

数学的には、この定理は次のように表せる。

Theorem 4.1 (Embedding of **CBVStep** into **Step**).

$M \longrightarrow_{\text{cbv}} N$ ならば、 $M \longrightarrow N$ である。

この定理は、**CBVStep** が **Step** の部分関係として見られることを意味する。すなわち、call-by-value の評価順序に従う一段階の簡約は、unrestricted な一段階の簡約としても妥当である。逆は一般には成り立たない。unrestricted **Step** は、call-by-value の評価順序に制限されていないため、**Step** による一段階簡約が常に **CBVStep** による一段階簡約であるとは限らない。

証明は、**CBVStep** の導出に関する場合分けによって行う。**CBVStep** の各 constructor は、対応する unrestricted **Step** の constructor に写される。具体的には、**CBVStep.beta** は **Step.beta** に、**CBVStep.appL** は **Step.appL** に、**CBVStep.appR** は **Step.appR** に対応する。また、条件分岐に関する **CBVStep.iteTrue**, **CBVStep.iteFalse**, **CBVStep.iteC** は、それぞれ対応する **Step** の規則に写される。

ここで重要なのは、**CBVStep.beta** が、引数が値であることを仮定する点である。一方、unrestricted **Step** の β 簡約は、引数が値であることを要求しない。したがって、**CBVStep.beta** の場合には、値性の仮定を用いずに **Step.beta** を得ることができる。この補題は、後に **CBVStep.preservation** を示す際に用いられる。既存の **subject_reduction** は unrestricted **Step** に対する preservation であるため、**CBVStep** による一步簡約を **Step** による一步簡約へ変換することで、既存の preservation 定理を再利用できる。

4.2 One-step CBV Preservation

次に、**CBVStep** に対する一步の preservation を示す。preservation は、型付けされた項が一步簡約された後も同じ型を持つことを主張する性質である。Lean 4 形式化では、この結果は **CBVStep.preservation** として与えられる。

```
theorem preservation {Γ : Context} {M N : Term} {A : Ty}
  (htype : Γ ⊢ M : A)
  (hstep : M →cbv N) :
  Γ ⊢ N : A
```

数学的には、この定理は次のように表せる。

Theorem 4.2 (One-step preservation for **CBVStep**).

$\Gamma \vdash M : A$ かつ $M \longrightarrow_{\text{cbv}} N$ ならば、 $\Gamma \vdash N : A$ である。

この定理は、**CBVStep** による一步簡約が型を保存することを意味する。すなわち、ある文脈 Γ の下で項 M が型 A を持つとき、 M を call-by-value の評価規則に従って一步簡約して得られる項 N も、同じ文脈の下で同じ型 A を持つ。

証明は、前節の定理 4.1 と、既存の **subject_reduction** を組み合わせることで得られる。まず、仮定 $M \longrightarrow_{\text{cbv}} N$ に対して定理 4.1 を適用し、 $M \longrightarrow N$ を得る。ここで得られた一步簡約は unrestricted **Step** による簡約である。したがって、既存の unrestricted **Step** に対する preservation である **subject_reduction** を、型付けの仮定 $\Gamma \vdash M : A$ と一步簡約 $M \longrightarrow N$ に適用することで、結論 $\Gamma \vdash N : A$ が従う。

Lean 4 上では、証明の本質は次の一行に集約される。

```
exact subject_reduction htype (to_step hstep)
```

ここで、**hstep** は $\Gamma \vdash M : A$ の証明であり、**hstep** は $M \longrightarrow_{\text{cbv}} N$ の証明である。また、**to_step hstep** によって、**CBVStep** による一步簡約が unrestricted **Step** による一步簡約へ変換される。その結果、既存の **subject_reduction** をそのまま適用できる。

このように、**CBVStep.preservation** は、preservation を一から証明し直すものではない。むしろ、既存の unrestricted **Step** に対する preservation を再利用し、

CBVStep.to_step を橋渡しとして用いることで得られる結果である。この点は、本研究における既存形式化の再利用の中心的な例である。

4.3 CBV Progress

次に、**CBVStep** に対する progress を示す。progress は、closed な well-typed term が、値であるか、あるいは一步計算を進められることを主張する性質である。本研究では、この性質を既存の call-by-value 一步簡約関係 **CBVStep** に対して形式化する。Lean 4 形式化では、この結果は **CBVStep.progress** として与えられる。

```
theorem progress {M : Term} {A : Ty}
  (htype : ([[] : Context] ⊢ M : A)) :
  IsValue M ∨ ∃ N, M →cbv N
```

数学的には、この定理は次のように表せる。

Theorem 4.3 (Progress for **CBVStep**).

$\emptyset \vdash M : A$ ならば、 M は値であるか、ある項 N が存在して $M \rightarrow_{\text{cbv}} N$ である。

この定理は、既存の **unrestricted Step** に対する progress から直接得られるものではない。既存の progress は、closed な well-typed term が値であるか、あるいは何らかの **unrestricted Step** によって一步進めることを示す。しかし、**unrestricted Step** による一步簡約は、必ずしも call-by-value の評価順序に従うとは限らない。したがって、**unrestricted progress** からは、ある項 N が存在して $M \rightarrow_{\text{cbv}} N$ であることは直ちには従わない。

そのため、本研究では、**CBVStep** の評価順序に沿って progress を別途証明する。証明は、型付け導出に関する場合分けによって行う。多くの場合は、値であることを示すか、対応する **CBVStep** の規則を適用することで進む。特に重要なのは、関数適用の場合と条件分岐の場合である。

関数適用 $M N$ の場合、まず関数部分 M に対して帰納法の仮定を用いる。 M が **CBVStep** によって一步進めるならば、**CBVStep.appL** によって適用全体 $M N$ も一步進める。一方、 M が値である場合、 M は関数型を持つ closed value であるため、補題 3.1 によって M はラムダ抽象であることが分かる。次に、引数部分 N に対して帰納法の仮定を用いる。 N が **CBVStep** によって一步進めるならば、**CBVStep.appR** によって適用全体 $M N$ も一步進める。 N も値であるならば、関数部分はラムダ抽象であり、引数部分は値であるため、**CBVStep.beta** によって call-by-value の β 簡約を行うことができる。

条件分岐

if M then N_1 else N_2

の場合、まず条件部 M に対して帰納法の仮定を用いる。 M が **CBVStep** によって一步進めるならば、**CBVStep.iteC** によって条件分岐全体も一步進める。一方、 M が値である場合、 M は型 **Bool** を持つ closed value であるため、補題 3.2 によって M は **true** または **false** のいずれかであることが分かる。 $M = \text{true}$ の場合には **CBVStep.iteTrue** によって then 分岐 N_1 へ進み、 $M = \text{false}$ の場合には **CBVStep.iteFalse** によって else 分岐 N_2 へ進む。

このように、**CBVStep.progress** の証明では、既存の canonical forms 補題と、**CBVStep** の評価規則を組み合わせる。特に、関数適用では「関数部分を先に評価し、関数部分が値になってから引数部分を評価する」という call-by-value の評価順序を反映している。また、条件分岐では、then 分岐や else 分岐を先に評価するのではなく、条件部を先に評価する。したがって、この定理は単なる **unrestricted progress** ではなく、call-by-value 評価順序に沿った progress を与えるものである。

4.4 Multi-step CBV Preservation

次に、**CBVStep** に対する一步の preservation を、多段階簡約へ拡張する。一步の preservation は、 $M \rightarrow_{\text{cbv}} N$ の場合に型が保存されることを示す。一方、実際の評価は一步だけ

でなく、複数回の一步簡約を繰り返すことで進む。そのため、**CBVStep** の反射推移閉包に対しても型が保存されることを示す必要がある。

Lean 4 形式化では、**CBVStep** の多段階簡約は **Rewriting.Star CBVStep** によって表される。本研究で追加した多段階 preservation は、**CBVStep.preservation_star** として与えられる。

```
theorem preservation_star {Γ : Context} {M N : Term} {A : Ty}
  (htype : Γ ⊢ M : A)
  (hsteps : Rewriting.Star CBVStep M N) :
  Γ ⊢ N : A
```

数学的には、この定理は次のように表せる。

Theorem 4.4 (Multi-step preservation for **CBVStep**).

$\Gamma \vdash M : A$ かつ $M \rightarrow_{\text{cbv}}^* N$ ならば、 $\Gamma \vdash N : A$ である。

この定理は、型付けされた項 M から **CBVStep** によって 0 回以上のステップで到達できる任意の項 N が、元の項と同じ型を持つことを主張している。特に、0 回の簡約の場合には $M = N$ であるため、型付けの仮定から直ちに結論が従う。1 回以上の簡約の場合には、一步ごとに定理 4.2 を適用することで、型が保存され続けることを示す。

証明は、**Rewriting.Star CBVStep M N** の導出に関する帰納法によって行う。反射の場合には、簡約列が 0 ステップであるため、仮定 $\Gamma \vdash M : A$ そのものが結論となる。推移的なステップの場合には、まず帰納法の仮定によって中間項の型付けを得る。そのうえで、一步の **CBVStep.preservation** を適用することで、次の項の型付けを得る。

したがって、**CBVStep.preservation_star** は、新しい preservation の証明を一から行うものではなく、一步の **CBVStep.preservation** を **Rewriting.Star** に沿って反復適用することで得られる結果である。この定理は、次節で述べる **CBVStep.type_safety** の証明において、到達先の項 N が依然として well-typed であることを示すために用いられる。

4.5 CBV Type Safety

最後に、**CBVStep** に対する type safety を示す。type safety は、closed な well-typed term から評価を開始したとき、評価途中で stuck state に到達しないことを表す性質である。本研究では、**Rewriting.Star CBVStep** によって到達可能な任意の項が、値であるか、あるいはさらに **CBVStep** によって一步進めることを示す。Lean 4 形式化では、この結果は **CBVStep.type_safety** として与えられる。

```
theorem type_safety {M N : Term} {A : Ty}
  (htype : ([] : Context) ⊢ M : A)
  (hsteps : Rewriting.Star CBVStep M N) :
  IsValue N ∨ ∃ P, N →cbv P
```

数学的には、この定理は次のように表せる。

Theorem 4.5 (Type safety for **CBVStep**).

$\emptyset \vdash M : A$ かつ $M \rightarrow_{\text{cbv}}^* N$ ならば、 N は値であるか、ある項 P が存在して $N \rightarrow_{\text{cbv}} P$ である。

この定理は、closed な well-typed term M から call-by-value 評価によって到達可能な任意の項 N が stuck していないことを主張する。すなわち、到達先の項 N は、すでに値であるか、そうでなければさらに call-by-value の一步簡約を進めることができる。

証明は、多段階 preservation と progress を組み合わせることで得られる。まず、仮定 $\emptyset \vdash M : A$ と $M \rightarrow_{\text{cbv}}^* N$ に対して、定理 4.4 を適用する。これにより、到達先の項 N も同じ型 A を持つこと、すなわち

$$\emptyset \vdash N : A$$

が得られる。

次に、定理 4.3 を項 N に適用する。 N は空文脈の下で型 A を持つ closed な well-typed term であるため、**CBVStep.progress** により、 N は値であるか、ある項 P が存在して $N \rightarrow_{\text{cbv}} P$ であることが従う。これは、定理 4.5 の結論そのものである。

この定理の結論には、 N の型付け $\emptyset \vdash N : A$ は明示的には含まれていない。しかし、その事実は定理 4.4 によって得られる。したがって、**CBVStep.type_safety** は、到達先の項が well-typed であることを多段階 preservation によって保証したうえで、progress を適用して no-stuckness を導く定理として理解できる。

以上により、本研究では、既存の **STLCextBool** 形式化に含まれる call-by-value 一步簡約関係 **CBVStep** に対して、一步の preservation, progress, 多段階 preservation, および type safety を Lean 4 上で補完した。特に、**CBVStep.to_step** によって既存の **unrestricted Step** に対する **subject_reduction** を再利用しつつ、call-by-value 評価順序に固有の progress を別途証明することで、既存の **CBVStep** に対する型安全性メタ理論を完成させた。

5 Discussion

本節では、本研究で追加した **CBVStep** に関する型安全性メタ理論について、証明構造上の特徴と限界を整理する。特に、既存の **unrestricted Step** に対するメタ理論をどのように再利用したか、また **CBVStep** に対する progress をなぜ別途証明する必要があったかを述べる。最後に、本研究で扱わない範囲と今後の課題について述べる。

5.1 Reusing Unrestricted Metatheory

本研究の特徴の一つは、既存の **unrestricted Step** に対するメタ理論を再利用して、**CBVStep** に対する preservation を導いている点である。**STLCextBool** 形式化には、**unrestricted Step** に対する preservation である **subject_reduction** が既存で含まれていた。一方で、**subject_reduction** は **unrestricted Step** に関する定理であり、そのままでは **CBVStep** に対して直接適用できない。

この間を接続するために、本研究では **CBVStep.to_step** を追加した。これは、任意の **CBVStep** による一步簡約が、**unrestricted Step** による一步簡約でもあることを示す補題である。この補題により、**CBVStep** による簡約を既存の **Step** による簡約へ変換できる。その結果、既存の **subject_reduction** をそのまま再利用して、**CBVStep.preservation** を導くことができる。

この証明構造は、既存形式化の上に新しいメタ理論を追加する際の再利用可能なパターンを示している。すなわち、新しい簡約関係が既存の簡約関係の部分関係として埋め込める場合、既存の preservation 定理を再利用できる可能性がある。本研究では、**CBVStep** が **unrestricted Step** の部分関係として見られることを明示することで、preservation の証明を重複させずに済ませている。

ただし、この再利用が可能なのは preservation に関する部分である。preservation は、一步簡約が型を保存するかどうかを問う性質であり、**CBVStep** の各一步簡約が **Step** の一步簡約でもあるなら、既存の **subject_reduction** によって型保存を得ることができる。一方、progress は「どの一步簡約が存在するか」を問う性質であるため、単に **CBVStep** が **Step** に埋め込めるという事実だけでは十分ではない。この点については次節で述べる。

5.2 Why CBV Progress Requires a Separate Proof

前節で述べたように、**CBVStep** に対する preservation は、**CBVStep.to_step** と既存の **subject_reduction** を組み合わせることで導くことができる。一方で、**CBVStep** に対する progress は、既存の **unrestricted Step** に対する progress から直接導くことはできない。これは、preservation と progress が異なる種類の性質を述べているためである。

preservation は、すでに与えられた一步簡約が型を保存するかどうかを問う性質である。したがって、**CBVStep** による一步簡約が **unrestricted Step** による一步簡約でもあることが分かれば、既存の **subject_reduction** を利用して型保存を示すことができる。これに対して progress は、closed な well-typed term が、値であるか、あるいは適切な一步簡約に進めることを示す性質である。つまり progress では、一步簡約が与えられているのではなく、一步簡約の存在そのものを示す必要がある。

既存の **unrestricted progress** は、closed な well-typed term が値であるか、あるいは **unrestricted Step** によって一步進めることを主張する。しかし、**unrestricted Step** による一步簡約が存在することは、**CBVStep** による一步簡約が存在することを意味しない。**unrestricted Step** は call-by-value の評価順序に制限されていないため、その一步簡約は **CBVStep** の規則に対応しない場合がある。

例えば、関数適用 $M N$ では、call-by-value 評価はまず関数部分 M を評価し、 M が値になった後で引数部分 N を評価する。さらに、 β 簡約は引数が値である場合にのみ行われる。したがって、関数適用の progress を示すには、関数部分と引数部分について、この評価順序に沿って場合分けを行う必要がある。単に $M N$ が何らかの **unrestricted Step** によって進めることが分かっても、それが call-by-value の次の一步であるとは限らない。

条件分岐の場合も同様である。

$$\text{if } M \text{ then } N_1 \text{ else } N_2$$

の call-by-value 評価では、then 分岐や else 分岐を先に評価するのではなく、まず条件部 M を評価する。条件部が true または false になったときに、対応する分岐へ簡約される。したがって、条件分岐の progress を示すには、条件部が一步進める場合と、条件部が値である場合を分け、後者では canonical forms によって true または false であることを示す必要がある。

このため、本研究では **CBVStep.progress** を、**CBVStep** の評価順序に沿って別途証明した。証明では、関数適用の場合に補題 3.1 を用いて関数部分がラムダ抽象であることを示し、条件分岐の場合に補題 3.2 を用いて条件部が true または false のいずれかであることを示す。このように、**CBVStep.progress** は、既存の **unrestricted progress** の単なる再利用ではなく、call-by-value 評価順序に固有の構造を反映した証明である。

5.3 Limitations

本研究の目的は、既存の **STLCextBool** 形式化に含まれる call-by-value 一步簡約関係 **CBVStep** に対して、型安全性メタ理論を補完することである。したがって、本研究の範囲は、**CBVStep** に対する preservation, progress, 多段階 preservation, および type safety に限定される。本節では、本研究で扱わない範囲を明確にする。

第一に、本研究は **CBVStep** を新たに定義するものではない。**CBVStep** は対象リポジトリに既存で定義されていた call-by-value 一步簡約関係である。本研究の貢献は、この既存の **CBVStep** に対して、型安全性に関する定理群を補完した点にある。

第二に、本研究は正規化を示すものではない。すなわち、任意の well-typed term が有限ステップで必ず値に到達することを主張するものではない。本研究で示す type safety は、評価によって到達可能な項が stuck しないことを示す性質であり、計算が必ず停止することを保証する性質ではない。

第三に、本研究は評価器の実装や実行可能なインタプリタの構成を目的とするものではない。本研究で扱うのは、**CBVStep** という関係として定義された操作的意味論と、それに関するメタ理論である。したがって、具体的な評価関数を定義し、その健全性や完全性を示すことは本研究の範囲外である。

第四に、本研究の progress 定理は、空文脈の下で型付けされた closed term を対象とする。これは、標準的な型安全性定理における progress の形に従うものである。open term に対する progress や、環境を伴う実行意味論については、本研究では扱わない。

第五に、本研究は、より大きな言語機能への一般化を主張するものではない。対象は **STLCextBool** に限定されており、例えば自然数型、リスト、レコード、例外、参照、あるいは一般再帰を含む体系については扱わない。これらの言語機能に対して同様の preservation, progress, type safety を形式化することは、今後の拡張課題である。

最後に、本研究は圏論的意味論や表示的意味論を形式化するものではない。本研究の主眼は、既存の call-by-value 操作的意味論に対する型安全性メタ理論を Lean 4 上で補完することにある。圏論的意味論との接続は重要な将来課題であるが、本論文では future work として位置づけるにとどめる。

5.4 Future Work

本研究では、既存の **STLCextBool** 形式化に含まれる call-by-value 一步簡約関係 **CBVStep** に対して、型安全性メタ理論を補完した。今後の課題としては、まず、この形式化をより大きな型付きラムダ計算へ拡張することが挙げられる。例えば、自然数型、リスト、レコード、あるいはその他の標準的な言語機能を追加した体系に対して、call-by-value の preservation, progress, および type safety を同様に形式化することが考えられる。

また、本研究で用いた証明構造を、他の簡約関係や他の体系に対して再利用できる形で整理することも重要である。特に、本研究では **CBVStep.to_step** によって既存の **unrestricted Step** に対する **subject_reduction** を再利用した。このように、ある簡約関係を既存の簡約関係へ埋め込むことで preservation を導くという方法は、他の評価戦略に対しても有効である可能性がある。一方で、progress については評価順序に依存した個別の議論が必要になるため、評価戦略ごとの証明パターンを整理することが今後の課題となる。

さらに、**STLCextBool** の型付け判断を、圏論的意味論の観点から解釈することも今後の重要な方向である。本研究では操作的意味論に基づく型安全性を扱ったが、型付きラムダ計算は圏論的には、型を対象、文脈を積、項を射として解釈することができる。この観点から、**STLCextBool** における真偽値型を $1 + 1$ として解釈し、true, false, および条件分岐を余積の構造として読むことが考えられる。

より広い方向としては、**STLCextBool** だけでなく、積型、和型、単位型、関数型を備えた **STLCext** 全体の圏論的意味論を整理することが挙げられる。この場合、関数型は指数対象、積型は圏論的積、和型は圏論的余積、単位型は終対象として解釈される。したがって、**STLCextBool** の真偽値型は、**STLCext** における余積構造の具体例として位置づけられる。

以上の方向は、本研究で扱った操作的メタ理論を、表示的意味論および圏論的論理学へ接続するための第一歩である。本論文では、これらの圏論的意味論を形式化することは扱わないが、今後の研究では、Lean 4 による型安全性形式化と、型付きラムダ計算の圏論的意味論との対応を明確にすることを目指す。

6 Conclusion

本研究では、Lean 4 Metatheory リポジトリに既存に含まれる **STLCextBool** 形式化を対象として、call-by-value 一步簡約関係 **CBVStep** に対する型安全性メタ理論を補完した。対象とする **CBVStep** は既存の形式化に含まれていたものであり、本研究はこれを新たに定義するものではない。本研究の貢献は、この既存の **CBVStep** に対して、preservation, progress, 多段階 preservation, および type safety を Lean 4 上で形式化した点にある。

まず、**CBVStep** による任意の一步簡約が **unrestricted Step** による一步簡約でもあることを **CBVStep.to_step** として示した。これにより、既存の **unrestricted Step** に対する preservation である **subject_reduction** を再利用し、**CBVStep.preservation** を導いた。この結果は、既存のメタ理論を再利用して call-by-value 簡約に対する型保存を得るものである。

一方で, **CBVStep** に対する **progress** は, 既存の **unrestricted progress** から直接得られるものではない. **unrestricted progress** は, **closed** な **well-typed term** が何らかの **unrestricted Step** に進めることを示すが, その一步簡約が **call-by-value** の評価順序に従うことまでは保証しない. そのため, 本研究では, **CBVStep** の評価順序に沿って **CBVStep.progress** を別途証明した. この証明では, 既存の **IsValue** および **canonical forms** 補題を用いて, 関数適用と条件分岐の場合を解析した.

さらに, **Rewriting.Star CBVStep** に基づいて, 多段階簡約に対する **preservation** である **CBVStep.preservation_star** を形式化した. 最後に, **CBVStep.preservation_star** と **CBVStep.progress** を組み合わせることで, **CBVStep.type_safety** を示した. これにより, **closed** な **well-typed term** から **call-by-value** 評価によって到達可能な任意の項は, 値であるか, さらに **CBVStep** によって一步進めることが分かる. したがって, 既存の **STLCextBool** における **call-by-value** 操作的意味論に対して, **stuck state** に到達しないことを **Lean 4** 上で形式化した.

今後の課題としては, より大きな型付きラムダ計算に対して同様の型安全性メタ理論を形式化することが挙げられる. また, 本研究で扱った **STLCextBool** の型付け判断を, 圏論的意味論の観点から解釈することも重要な方向である. 特に, 真偽値型を $1+1$ として解釈し, さらに積型, 和型, 単位型, 関数型を備えた **STLCext** 全体の圏論的意味論へ拡張することは, 操作的メタ理論と表示的意味論を接続する今後の研究課題である.

References

- [1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, Automated Deduction – CADE 28, volume 12699 of Lecture Notes in Computer Science, pages 625–635. Springer, 2021.
- [2] Benjamin C. Pierce. Types and Programming Languages. MIT Press, 2002.
- [3] Benjamin C. Pierce et al. Software foundations. <https://softwarefoundations.cis.upenn.edu/>, 2026. Electronic textbook series; accessed 2026-07-09.
- [4] Arthur Ramos and contributors. Metatheory: A Lean 4 formalization library for programming language metatheory. <https://github.com/Arthur742Ramos/Metatheory>, 2026. GitHub repository; accessed 2026-07-09.
- [5] Arthur Ramos, Anjolina Oliveira, Ruy de Queiroz, and Tiago de Veras. A modular Lean 4 framework for confluence and strong normalization of lambda calculi with products and sums, 2025.
- [6] Philip Wadler, Wen Kokke, and Jeremy G. Siek. Programming language foundations in Agda. <https://plfa.inf.ed.ac.uk/20.07/>, 2020. Version 20.07; accessed 2026-07-09.
- [7] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. Information and Computation, 115(1):38–94, 1994.